AC-II

Aula 1

- microprocessadores vs microcontroladores
- sistemas embebidos
- desenvolvimento de aplicações para microcontroladores
- noção de periférico; estrutura básica de um modulo de I/O;
  registos de controlo; status e dados; modelo de programação

microprocessador vs microcontrolador

- circuito integrado com um ou mais CPU;
- não tem memória interna (além dos registos)
- os barramentos estão disponíveis no exterior
- é necessário adicionar RAM, ROM e periféricos para ficar completo
- opera em frequências altas (>3Ghz)
- Sistemas computacionais de uso geral

- circuito integrado que inclui CPU, RAM, ROM e periféricos
- funciona em frequências baixas
- baixo consumo de energia
- grande variedade de periféricos e interfaces com o exterior
- rapidez de resposta a eventos externos (sistemas de tempo real)
- utilizado em tarefas específicas
  Ex. controlo da velocidade de um motor)

Sistemas embebidos

- sistema computacional especializado
  realiza algo específico ou controla algo
- tem requisitos próprios e executa apenas tarefas pré-definidas
- recursos disponíveis mas mais limitados que num sistema computacional de uso geral (menos memória,...)
- custo inferior a um sistema computacional de uso geral

Ex. de utilização: eletrónica de consumo, automóveis
telecomunicações, domótica, robótica, iot

microcontrolador

dispositivo programável que integra num único circuito:

- Unidade de Processamento
- Memoria Volátil e Não Volátil
- Portos I/O (E/S)

e ainda inclui, timers, conversor A/D, Serial I/O.

barramentos (dados, endereços e controlo) interligam todos (não estão geralmente acessíveis externamente

versatilidade

externamente há, em geral, pinos que podem ser configurados

geração do código

em C ou assembly

é necessário utilizar cross-compilers / cross-assemblers

no desenvolvimento de aplicações, devido a recursos necessários

programa-monitor: é um programa que reside, de forma permanente, na memória não volátil

bootloader: é um programa que reside, de forma permanente, na memória do microcontrolador e que disponibiliza funções básicas de transferência e execução de um programa

in-circuit debugger (ICD): é um dispositivo hardware controlado por software do host que permite a transferência e execução controlada de um programa.

O ICD é normalmente, necessário para a transferência de um programa-monitor ou bootloader

tecnicas de memória não volátil

ROM - programada no processo de fabrico

PROM - programável uma única vez

EPROM - escrita em segundos, apagamento em minutos

EEPROM - o apagamento e a escrita podem ser efetuadas no mesmo circuito
o apagamento é feito byte a byte
escrita muito mais lenta que leitura

Flash EEPROM - semelhante a ↑
escrita pressupõe a inicialização prévia das zonas de memória a escrever (reset)
tecnologia mais rápida porque o reset pode ser feito em blocos.

O nosso microcontrolador

memória flash: 512 kB

memória RAM: 128 kB

Versão 64 pinos (também há 100 e 121)

CPU

32 registos de 32 bits

espaço de endereçamento de 32 bits

organização de memória: byte-addressable

max. freq. 80MHz

Microcontroladores baseados em Arquitetura de Harvard

→ dois espaços de endereçamento independentes: um para instruções, outro para dados

→ apenas o bloco ID tem acesso à memória

→ o programa não pode aceder a dados armazenados na memória de instruções

→ o CPU não pode executar instruções armazenadas na memória de dados

Solução PIC32 para limitações da Arquitetura de Harvard

Bus Matrix

é um comutador (switch) que funciona à mesma frequência do CPU

estabelece ligações ponto a ponto

entre o bloco IF do CPU e memória Flash

entre o bloco IF e a memória RAM

entre o bloco MEM do CPU e memória RAM

entre o bloco MEM do CPU e memória Flash

Assim, o espaço de endereçamento aparece como um espaço linear unificado (instruções e dados residem no mesmo espaço de endereçamento, cada um ocupando uma gama de endereços única)

Ou seja, para o programador é uma Arquitetura Von Neumann

dispositivos periféricos

estrutura básica de um módulo de I/O

registos de controlo, status e dados

modelo de programação

- assíncronos relativamente ao CPU
- geram diferentes quantidades de informação
  com diferentes formatos a diferentes velocidades
- mais lentos que o CPU e memória

é necessária uma interface que adapte características
do dispositivo periférico às do CPU/memória

modelo de programação do periférico

Data Register (Read/Write)

Status Register (Read only)

Control Register (Write only ou Read/Write)

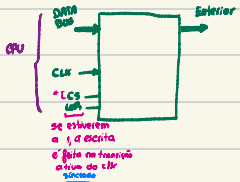
Comunicação entre CPU e outros dispositivos

protocolo
- Write (CPU → dispositivo externo)
- Read (CPU ← dispositivo externo)

aula 2

1 exemplo de periféricos simples
2 portos I/O no PIC32
3 transferência de informação entre periféricos e memória (polling)

① porto de saída de 32 bits

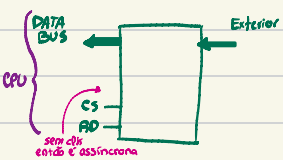


- armazena informação vinda do CPU durante escrita na memória (sw)
  MEM
- o sinal WR é gerado pelo descodificador de endereços: fica ativo se o endereço gerado pelo CPU for igual ao endereço do porto
  → flip-flops armazenam os dados na transição ascendente do clock

se estiverem a 1, a escrita é feita na transição ativa do clk gerado

porto de entrada de 32 bits

- em geral portos de entrada não têm capacidade de armazenamento)



- a informação presente na entrada é transferida para o CPU durante uma operação de leitura
- as saídas têm portas tri-state, só são ativas se CS e RD estão ativos
  resistor

② o microcontrolar PIC32... disponibiliza vários portos I/O (de B a G) com várias dimensões, mas no máximo 16 bits

- cada um destes pode ser configurado como Input ou Output
- um porto I/O de n bits é um conjunto de n portos de I/O de 1 bit

Cada um dos portos tem associado 12 registos de 32 bits, os que são usados em AC-II são:

- TRIS - configuração → bit n = 1, bit n configurado como entrada; bit n = 0, bit n configurado como saída
- PORT - leitura
- LAT - escrita

} 32 bits cada registo em que os 16 mais significativos são ignorados

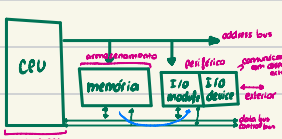
Detalhes

- cada porto inclui uma porta schmitt trigger (comparador com histerese) para melhorar a imunidade ao ruído
- no porto de entrada, o sinal externo é sincronizado através de dois flip-flops (S1 e S2) para resolver problemas de meta-estabilidade decorrentes pelo sinal externo ser assíncrono ao clk do CPU
- os dois flip-flops impõem um atraso de dois ciclos de relógio na propagação do sinal até ao barramento de dados do CPU

número de ciclos de relógio gastos pelo CPU em tarefas que não estão diretamente relacionadas com a transferência

o overhead deste método depende do número de vezes que é executado o polling

③



1. transferência de informação disco → memória
   1º efetuar o pedido
   2º esperar que o disco tenha a informação disponível
   3º transferir a informação da memória
2. Estes valores dependem da tecnologia do disco:
   Latência: tempo que decorre do pedido ao 1º byte de informação
   Taxa de transferência de pico (burst): nº máximo de bytes transferidos por segundo após latência
   Taxa de transferência: nº total de bytes transferidos / tempo total (incluindo latência)

3. técnicas de gestão de transferência de dados
   - E/S programada (programmed I/O)
     - CPU assume controlo total da transferência
     - CPU inicia operação, verifica repetidamente se o dispositivo está pronto para enviar/receber (polling) e efetua a transferência
     - polling: CPU fica em "espera ativa", pode desperdiçar ciclos de CPU

técnica básica que pode ser utilizada quando a velocidade do periférico não diminui drasticamente a capacidade de processamento do CPU

- E/S por interrupção (interrupt driven I/O) ← solução do polling
  - o periférico sinaliza que está pronto
  - o CPU executa a transferência de dados enquanto aguarda a interrupção pode continuar a fazer outras tarefas
- E/S por acesso direto à memória (DMA)
  - um dispositivo externo ao CPU (DMA) assegura a transferência dos dados diretamente entre a memória e o periférico

Aula 3 e 4

1 Transferência de informação entre os periféricos e a memória
- 1.1 E/S por interrupção (interrupt driven I/O)
- 1.2 E/S por acesso direto à memória

1.1

o periférico sinaliza o CPU quando está pronto para trocas dados

depois de sinalizado, o CPU suspende temporariamente a execução do programa em curso para executar a rotina de tratamento correspondente
essa rotina chama-se rotina de serviço à interrupção (RSI) ou interrupt handler

a transferência é feita pelo CPU

esta técnica mascara o problema de latência

exemplo: leitura de dados de um periférico

1º CPU envia pedido de informação ao periférico (registo de controlo do periférico)
1º continua as suas tarefas
2º quando tiver disponível, o periférico gera um pedido de interrupção ao cpu
4º CPU atende a interrupção
- Suspende a execução do programa corrente
- Salta para a rotina de atendimento à interrupção (interrupt handler) que transfere a informação
- Retoma a execução do programa suspenso

Características
Não existe qualquer ciclo de espera
o programa em execução pode ser interrompido a qualquer momento
A RSI tem que salvaguardar o contexto do programa (registos internos, ...)
O contexto guardado tem de ser reposto antes de terminar a RSI
A palavra-chave interrupt distingue uma função do tipo RSI de uma função normal

exceções e interrupções

eventos que, não sendo branches ou jumps, alteram o fluxo normal de execução

- → eventos com origem no CPU, inesperados e decorrentes da execução das próprias instruções] exceções
  Ex. divisão por zero, overflow, acesso a endereços inválidos
- → eventos com origem externa ao CPU que surgem assincronamente com o funcionamento deste] interrupções
  Ex. quando é premida uma tecla do teclado

exceções: a instrução que a gera não termina, passa logo para a rotina de tratamento da exceção
interrupções: a unidade de controlo verifica se há algum pedido de interrupção pendente antes de iniciar o fetch de uma nova instrução

Secção crítica:

Uma sequência de instruções em que a ocorrência de uma interrupção causa um comportamento incorreto

Processamento de interrupções pelo CPU

1. identificação da fonte de interrupção e obtenção do endereço da RSI
2. salvaguarda do contexto atual da CPU (valor corrente do PC e de flags de estado associados ao sistema)
3. desativação das interrupções
4. carregamento do PC com o endereço da RSI
5. Execução da RSI até encontrar a instrução de retorno
6. Execução da instrução de retorno da RSI
   reposição do salvaguardo

Latência da interrupção: tempo que decorre desde a intervenção até à primeira instrução da RSI
Overhead: causado pela mudança de contexto

→ salvaguardar, repor o contexto salvaguardado - estas 2 operações requerem, no mips, 30 instruções

Ações gerais que devem ser implementadas à RSI

1. Salvaguarda do programa que foi interrompido → registos internos do CPU → memória (stack)
2. ações associadas ao processamento da interrupção → código
3. Reposição do contexto do programa interrompido → registos internos do CPU ← memória (stack)
4. Conclusão da RSI com a instrução de retorno ("Return from Interrupt/exception". "eret" no caso do MIPS)

Num sistema real é comum que vários periféricos gerem interrupções para organizar e priorizar esses pedidos:

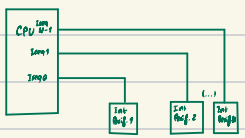
- multi-filas (linhas de interrupção)
- identificação da fonte de interrupção por software
- interrupções vetorizadas
  - identificação do periférico que gerou a interrupção é feita por hardware

Num sistema real vários periféricos podem gerar interrupções

como lidar?

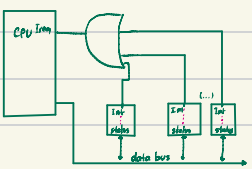
- → múltiplas linhas de interrupção
  cada periférico tem a sua linha dedicada
- → identificação da fonte de interrupção por software
- → interrupções vetorizadas

1. múltiplas linhas



- identificação automática da fonte
- uma RSI para cada fonte
- número máximo de fontes é igual ao número de linhas de interrupção da CPU
- cada linha tem uma prioridade (no hardware pode ser utilizada prioridade fixa ou rotativa)
  O CPU atende quem tem maior prioridade

2. identificação da fonte

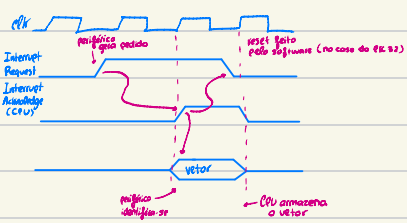


- apenas uma entrada de interrupção e uma RSI para todas as fontes
- A RSI lê o registo status de cada um dos periféricos até encontrar um que tenha gerado um pedido de interrupção
- no caso de pedidos simultâneos, a ordem pela qual os periféricos são "verificados" determina a prioridade

interrupções vetorizadas

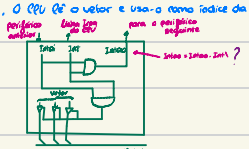
- → CPU tem apenas 1 entrada de interrupção
- → a identificação é feita por hardware
- → cada periférico possui um id, designado por vetor (identificador único)
- → uma RSI por cada vetor de interrupção
- → o periférico identifica-se pelo seu vetor, na fase de atendimento
- → o vetor vai ser usado como índice de uma tabela que contém:
  - endereço de cada RSI ou
  - instruções de salto incondicional para as RSI

A identificação da fonte é feita por hardware pelo processo "Interrupt acknowledge cycle"



daisy chain

1. CPU deteta pedido ('Ireq') e está em condições de o atender, ativa 'Inta' Interrupt Acknowledge
2. O sinal 'Inta' percorre a cadeia até ao periférico que gerou interrupção
3. O periférico coloca o seu id/vetor no barramento de dados e bloqueia propagação do sinal Interrupt Acknowledge
4. O CPU lê o vetor e usa-o como índice da tabela (mencionada atrás)

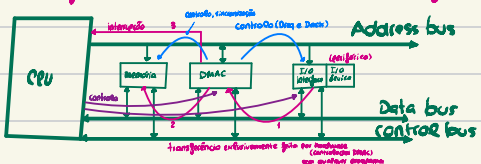


Transferência por Acesso Direto à Memória (DMA)

→ O problema da longa latência apresentada pelos periféricos é tratada pela técnica E/S por interrupção mas essa técnica não resolve problemas de transferências a taxas elevadas, uma vez que o limite é sempre imposto pelo facto de o CPU ter que executar um programa para efetuar a transferência

Por exemplo:
transferir dados de um periférico para a memória, temporal:
- → tempo de latência de resposta + tempo de execução do prólogo
- → tempo de execução de instruções
- → tempo necessário para verificar se todos os dados foram transferidos

Solução ⟶ DMA (Direct Memory Access)
consiste na transferência de informação do periférico diretamente para a memória (ou o contrário) sem intervenção do processador
um periférico especializado (DMAC - DMA Controller) efetua a transferência



1. transferência de dados
2. DMAC escreve os dados na memória
3. depois da transferência terminar DMAC envia interrupção ao CPU

?

DMAC
- tem registos internos
  - endereço origem da informação (ativa)
  - endereço destino da informação (escrita)
  - quantidade de informação a transferir (nº de bytes/words)

  } CPU pode
- também pode ter registos sobre o estado da transferência

O controlador DMA é um periférico especializado na transferência de dados (cópia), de forma autónoma, em hardware, após programação feita em CPU] configura o DMAC
o DMAC não executa programas
durante a transferência DMAC tem a capacidade de controlar os barramentos de endereços, dados e controlo, como se fosse o CPU

Para efetuar transferências de dados o DMA:
- → lê uma palavra do dispositivo fonte para um registo interno - fetch
- → escreve a palavra guardada no registo interno no dispositivo destino - deposit
- → incrementa source address e destination address
- → incrementa o número de palavras transferidas
- → se não transferiu tudo, repete desde o início

Para isto o DMAC controla os barramentos de endereços e os sinais rd e wr

Apenas um dos dois, DMAC e CPU, pode estar ativo, por defeito é o CPU
ou seja, só um bus master

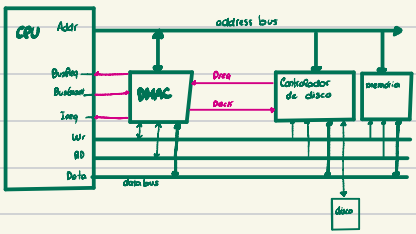
Para o DMAC se tornar bus master o DMAC:
1. ativa o sinal BusReq (Bus Request)
2. espera pela ativação do sinal BusGrant (CPU ativa o BusGrant quando está em condições de libertar os barramentos)

Só depois disto, pode efetuar a transferência
e assume o controlo até todos os dados serem transferidos] bloco

Cycle Stealing: o DMAC assume o controlo durante 1 bus cycle e liberta-os de seguida - transfere parcialmente 1 palavra

Bus cycle: sequência fundamental de passos necessários para executar uma única operação de transferência de dados (leitura ou escrita) entre dois componentes do sistema (por exemplo, CPU e memória)
um bus cycle é quantificado em ciclos do relógios de sincronização

DMA no Hardware



modo bloco

exemplo de uma transferência

1. CPU envia um comando ao controlador do disco (DiskCtrl)
   leitura de um dado setor, número de palavras
2. CPU programa DMAC: endereço inicial da transferência
   endereço inicial do destino
   número de palavras a transferir
3. CPU continua com outras tarefas
4. Quando o DiskCtrl tiver lido a informação pedida para a memória interna ativa o sinal Dreq do DMAC
5. O DMAC ativa o sinal BUSReq
6. CPU coloca barramentos em alta impedância e ativa o sinal BusGrant
7. O DMAC ativa o sinal DACK e o DiskCtrl, em resposta, desativa o sinal Dreq
8. DMAC efetua a transferência
   i) lê endereço-origem para um registo interno
   ii) escreve do registo interno para o endereço destino (incrementa endereços e número de palavras transferidos)
   Durante esta fase, o CPU está impedido de aceder à memória de dados
9. DMAC termina a transferência:
   - desativa o sinal Dack
   - deixa de controlar os barramentos (passam a ser entrados)
   - desativa o sinal BusReq
   - ativa o sinal de interrupção
10. CPU desativa BusGrant e pode novamente usar os barramentos
11. CPU atende a interrupção gerada pelo DMAC

Cycle Stealing

repete:
1. toma-se bus master — lê 1 palavra de source address
2. liberta os barramentos — espera durante um tempo
3. toma-se bus master — escreve 1 palavra no destination address
4. liberta os barramentos — incrementa source/destination address, nº de bytes/words transferidos

Se ainda não transferiu o que devia

DMAC dedicado → controlador de DMA do periférico

Para a transferência completa de 1 palavra o barramento é usado 1 vez (1 "bus cycle" por palavra)

Aula 5

- Timers
- Watchdog timer
- Estudo de caso: Timers do PIC32

Um timer é um dispositivo periférico que permite a medição de tempo partindo de uma referência temporal conhecida

exemplo: geração de um impulso com uma duração definida, Δt

exemplo: geração de um impulso com um período definido, T

$$\text{duty cycle} = \frac{t_{on}}{T} \cdot 100\ (\%)$$

(t_{on}: tempo em 1)

a possibilidade de alterar t_{on} sem alterar o valor T é útil e chama-se PWM (Pulse Width Modulation)

modulação por largura de pulso

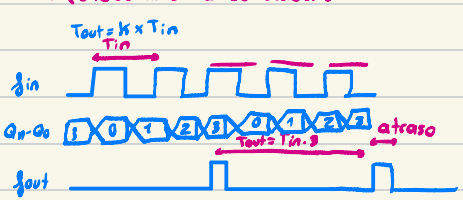
o funcionamento dos timers baseia-se na contagem de ciclos de um sinal de relógio com frequência conhecida

Divisão de frequência

- o fator de divisão é fixo
  2, 4, 8, 16, ...
- $f_{out} = \frac{f_{in}}{K}$

reset do contador assíncrono: o tempo a 1 do sinal R não é controlado (2 atrasos de propagação)

O período do sinal de saída é



O muda sem relógio

reset do contador síncrono: tempo a 1 sinal de saída é 1 ciclo de relógio do sinal de entrada

$T_{out} = (K+1) \times T_{in}$

O espera pelo relógio

O timer tem que permitir o controlo simultâneo do período do sinal de saída e do tempo durante o qual esse sinal está a 1

Watchdog timer

monitoriza a operação do microprocessador e, em caso de falha, força o seu reinício

reset da contagem feito dentro do ciclo

se houver uma falha que implique a quebra do ciclo a função do clearWatchdogTimer() deixa de ser executada e o watchdog timer deixa de ser reiniciado
→ quando o contador do watchdog chega ao maximo, força um reset ao microprocessador

Timers do PIC32

5 timers de 16 bits (T1, T2, T3, T4 e T5)

T2, T3, T4 e T5 timers de tipo B
podem ser agrupados em pares
T2 e T3 e/ou T4 e T5

T1 é timer tipo A
A frequência é dada pelo Peripheral Bus Clock

Na placa DETPIC32 a frequência de PBClk é metade da frequência do CPU, 20MHz

não tem saída acessível no exterior

Tipo A

$$f_{outpresc} = \frac{PBCLK}{K_{prescaler}}$$ ← entrada no contador de 16 bits

$$f_{out} = \frac{f_{outpresc}}{PR+1}$$ ← saída do timer

Tipo B

$$f_{outpresc} = \frac{PBCLK}{K_{prescaler}}$$ ← entrada no contador de 16 bits

$$f_{out} = \frac{f_{outpresc}}{PR_{2,4}+1}$$ ← saída do timer

exercícios

calcular as constantes para gerar um sinal pwm com frequência 8Hz e um duty cycle de 20%, usando T2 como referência e OC1 como saída (PBCLK=20MHz

$$f_{out} = \frac{f_{outpresc}}{PR2+1} = \frac{\frac{PBCLK}{K_{prescaler}}}{PR2+1}$$

$$K_{prescaler} = \frac{PBCLK}{(PR2+1) f_{out}}$$

1. Cálculo da constante

$$K_{prescaler} \geq \left[\frac{PBCLK}{(65535+1)\times 8}\right] = 39$$

$K_{prescaler} = 64$

Aula 6

→ endereçamento dos unidades I/O

→ descodificação de endereços e geração de sinais de seleção de memoria

→ exemplo de um gerador de sinais de seleção com subdivisão da gama de endereços

Uma operação de acesso ao cpu a um dispositivo externo envolve usar:

- barramento de endereços para especificar o endereço do dispositivo a aceder
- barramento de controlo (read ou write)
- barramento de dados (transportar informação)

Partilha do barramentos de dados

descodificação de endereços
→ restantes dispositivos em Tri-State (alta impedância)

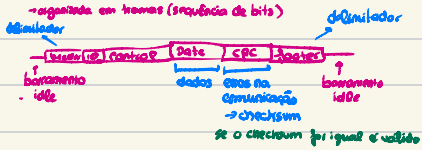
Descodificação total
→ apenas um endereço possível para acesso
→ todos os bits adequados são descodificados

Descodificação parcial
→ vários endereços para aceder à mesma posição de memória/registo de periférico
→ apenas alguns bits são descodificados
→ circuitos de descodificação mais simples e menores atrasos

aula 7

- transmissão de dados

1. transmissão de dados orientada ao bit
   → organizada em frames (sequência de bits)



* o checksum for igual é válido

topologias

half-duplex

- a comunicação ocorre num único sentido

device driver: entre a interface série e o software configurações...

Tipos de comunicação

Simplex: comunicação apenas num sentido
(transmissor → recetor) → telemetria
leitura remota de sensores

full-duplex: comunicação bidirecional simultânea

half-duplex: comunicação bidirecional (nos dois sentidos), mas não simultaneamente